

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)



PROJECT REPORT

TETRIS GAME

COURSE:

DIGITAL SYSTEM DESIGN

SUBMITTED TO:

Dr. M AKMAL SHAFIQUE / LE USAMA SHOUKAT

SUBMITTED BY:

1:	NOSHEEN FATIMA	411289
2:	UJALA KIRAN	432071
Degree		44
Syndicate		A

Date of Submission:

23rd December 2025.

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

Contents

Objectives:	3
Hardware/ Software required:	3
Software:	3
Purpose:	3
Usage in Lab:	Error! Bookmark not defined.
PROJECT OVERVIEW AND LOGIC:	3
Overall Working Logic:	4
1) Clock and Timing Logic	4
2) Game State and Control Logic.....	4
3) Block and Board Representation	5
4) Random Block Generation.....	5
VGA Display Logic:	5
Seven Segment Display Logic:	6
Solution:	6
Verilog Module:	6
Implementation Constraints File:.....	44
State Machine Diagram:.....	46
Test Bench:	46
Output:	51
Simulation:	51
FPGA Implementation:	52
Conclusion:	52

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

Objectives:

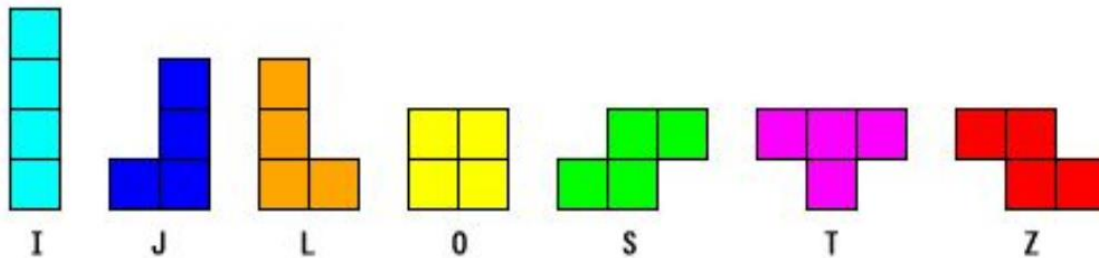
The objective of this project is to implement the classic Tetris game on the Nexys 3 FPGA using Verilog, gaining hands-on experience in digital system design. The project focuses on using finite state machines for game control, VGA interfacing for graphical output, and clock division for smooth gameplay. It also includes managing block representation, collision detection, random block generation, and displaying the score on seven-segment displays, providing practical experience in FPGA design, simulation, and testing.

Hardware/ Software required

Hardware: FPGA board, PC

Software: Xilinx ISE Design Suite

PROJECT OVERVIEW AND LOGIC:



The seven game pieces, called tetrominoes, and the letters commonly used to describe their shapes.

PROJECT OVERVIEW:

In this project, we implemented a basic version of the classic Tetris game on the Nexys 3 FPGA board. The game is displayed using VGA output and controlled through on-board buttons and switches. The purpose of this project was to understand digital system design, finite state machines, VGA interfacing, and hardware-based game logic using Verilog.

The game board is a 10×22 grid where different Tetris blocks (tetrominoes) fall from the top. The user can move, rotate, drop, and pause the game using buttons. The score is displayed on the seven-segment display.

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

Overall Working Logic:

The complete project works by dividing the design into **small and manageable modules**, where each module performs a specific task. All modules are connected in the top-level `tetris` module.

The main logic of the project can be explained in four parts:

1. Clock and timing control
2. Game logic and state control
3. VGA display logic
4. Seven segment score display

1) Clock and Timing Logic

The Nexys 3 board provides a 100 MHz clock, which is too fast for VGA timing and reliable logic operation. Therefore, the clock is divided by 4 to generate a **25 MHz clock**, which is used throughout the design.

This 25 MHz clock is suitable for:

- VGA display timing
- Game logic updates
- Seven-segment display multiplexing

A separate **game clock** is generated to make the falling speed of blocks equal to **1 block per second**. This makes the game playable and realistic.

2) Game State and Control Logic

The game behavior is controlled using a **finite state machine (FSM)**. The main states include:

- **IDLE**: Waiting for reset to start the game
- **PLAY**: Block falls normally and user can control it
- **PAUSE**: Game is paused
- **DROP**: Block drops instantly to the bottom
- **SHIFT**: Completed rows are removed and upper rows shift down

Using states makes the design organized and avoids complex combinational logic.

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

3) Block and Board Representation

The game board is represented using a **220-bit vector**, where each bit corresponds to one position on the 10×22 board.

- 1 means a block is present
- 0 means the position is empty

The current falling block is stored separately using:

- Block type
- X and Y position
- Rotation value

Collision detection is performed before moving or rotating a block to ensure it does not go outside the screen or overlap with fallen blocks.

4) Random Block Generation

A randomizer module is used to select the next tetromino. The randomness comes from the clock and unpredictable user input timing. This makes the sequence of blocks appear random to the player.

VGA Display Logic:

The VGA controller generates:

- `hsync` and `vsync` signals for screen timing
- `rgb` signals for pixel color

The screen resolution used is **640 × 480**.

- Background is black
- Game board background is gray
- Border is white
- Fallen blocks are white
- Current falling block is colored

Since VGA sends **one pixel at a time**, RGB values change continuously. This behavior looks repetitive in simulation but works correctly on actual hardware.

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

Seven Segment Display Logic:

The score is shown on the four-digit seven-segment display.

- The score increases by **1 for each completed row**
- Maximum score is 9999

Because all digits share the same segment lines, **multiplexing** is used. Each digit is enabled one at a time at high speed, making it appear that all digits are ON simultaneously.

Solution:

Verilog Module:

```
`timescale 1ns / 1ps
// Complete corrected Tetris (single-file) for FPGA
// - Assumes incoming clk_too_fast = 100 MHz
// - Generates pixel clock ~25 MHz
// - Debouncer / game timing adjusted for 25 MHz
// - Safe indexing for fallen_pieces
//
// Top module: tetris

// The width of the screen in pixels
`define PIXEL_WIDTH 640
// The height of the screen in pixels
`define PIXEL_HEIGHT 480

// Used for VGA horizontal and vertical sync
`define HSYNC_FRONT_PORCH 16
`define HSYNC_PULSE_WIDTH 96
`define HSYNC_BACK_PORCH 48
`define VSYNC_FRONT_PORCH 10
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
`define VSYNC_PULSE_WIDTH 2
`define VSYNC_BACK_PORCH 33

// How many pixels wide/high each block is
`define BLOCK_SIZE 20

// How many blocks wide the game board is
`define BLOCKS_WIDE 10

// How many blocks high the game board is
`define BLOCKS_HIGH 22

// Width of the game board in pixels
`define BOARD_WIDTH (`BLOCKS_WIDE * `BLOCK_SIZE)
// Starting x pixel for the game board (centered)
`define BOARD_X (((`PIXEL_WIDTH - `BOARD_WIDTH) / 2))

// Height of the game board in pixels
`define BOARD_HEIGHT (`BLOCKS_HIGH * `BLOCK_SIZE)
// Starting y pixel for the game board (centered)
`define BOARD_Y (((`PIXEL_HEIGHT - `BOARD_HEIGHT) / 2))

// The number of bits used to store a block position
`define BITS_BLK_POS 8
// The number of bits used to store an X position
`define BITS_X_POS 4
// The number of bits used to store a Y position
`define BITS_Y_POS 5
// The number of bits used to store a rotation
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
`define BITS_ROT 2

// The number of bits used to store how wide / long a block is (max
of decimal 4)

`define BITS_BLK_SIZE 3

// The number of bits for the score. The score goes up to 10000

`define BITS_SCORE 14

// The number of bits used to store each block

`define BITS_PER_BLOCK 3


// The type of each block
`define EMPTY_BLOCK 3'b000
`define I_BLOCK 3'b001
`define O_BLOCK 3'b010
`define T_BLOCK 3'b011
`define S_BLOCK 3'b100
`define Z_BLOCK 3'b101
`define J_BLOCK 3'b110
`define L_BLOCK 3'b111


// Color mapping (8-bit packed)
`define WHITE 8'b11111111
`define BLACK 8'b00000000
`define GRAY 8'b10100100
`define CYAN 8'b11110000
`define YELLOW 8'b00111111
`define PURPLE 8'b11000111
`define GREEN 8'b00111000
`define RED 8'b00000111
`define BLUE 8'b11000000
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
`define ORANGE 8'b00011111

// Error value
`define ERR_BLK_POS 8'b11111111

// Modes
`define MODE_BITS 3
`define MODE_PLAY 0
`define MODE_DROP 1
`define MODE_PAUSE 2
`define MODE_IDLE 3
`define MODE_SHIFT 4

// The maximum value of the drop timer
`define DROP_TIMER_MAX 10000

// calc_cur_blk: compute flattened block positions and size

module calc_cur_blk(
    input wire [`BITS_PER_BLOCK-1:0] piece,
    input wire [3:0] pos_x,
    input wire [4:0] pos_y,
    input wire [1:0] rot,
    output reg [7:0] blk_1,
    output reg [7:0] blk_2,
    output reg [7:0] blk_3,
    output reg [7:0] blk_4,
    output reg [2:0] width,
    output reg [2:0] height
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
);

always @ (*) begin
    // default
    blk_1 = `ERR_BLK_POS;
    blk_2 = `ERR_BLK_POS;
    blk_3 = `ERR_BLK_POS;
    blk_4 = `ERR_BLK_POS;
    width = 0;
    height = 0;

    case (piece)
        `EMPTY_BLOCK: begin
            // Already defaults to ERR
        end
        `I_BLOCK: begin
            if (rot == 0 || rot == 2) begin
                blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
                blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
                blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x;
                blk_4 = ((pos_y + 3) * `BLOCKS_WIDE) + pos_x;
                width = 1;
                height = 4;
            end else begin
                blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
                blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
                blk_3 = (pos_y * `BLOCKS_WIDE) + pos_x + 2;
                blk_4 = (pos_y * `BLOCKS_WIDE) + pos_x + 3;
                width = 4;
            end
        end
    endcase
end
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        height = 1;
    end
end
`O_BLOCK: begin
    blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
    blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
    blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
    blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x + 1;
    width = 2;
    height = 2;
end
`T_BLOCK: begin
    case (rot)
        0: begin
            blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
            blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
            blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;
            blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 2;
            width = 3;
            height = 2;
        end
        1: begin
            blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
            blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
            blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x;
            blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;
            width = 2;
        end
    end
end
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        height = 3;
    end
    2: begin
        blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
        blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
        blk_3 = (pos_y * `BLOCKS_WIDE) + pos_x + 2;
        blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;

        width = 3;
        height = 2;
    end
    3: begin
        blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
        blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;

        blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x
+ 1;

        blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
        width = 2;
        height = 3;
    end
endcase
end
`S_BLOCK: begin
    if (rot == 0 || rot == 2) begin
        blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
        blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x + 2;
        blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
        blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x + 1;
        width = 3;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        height = 2;
    end else begin
        blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
        blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
        blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x + 1;
        blk_4 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x + 1;
        width = 2;
        height = 3;
    end
end
`Z_BLOCK: begin
    if (rot == 0 || rot == 2) begin
        blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
        blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
        blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x + 1;
        blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x + 2;
        width = 3;
        height = 2;
    end else begin
        blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
        blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
        blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x;
        blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x + 1;
        width = 2;
        height = 3;
    end
end
end
`J_BLOCK: begin
    case (rot)
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
0: begin
    blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
    blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;

    blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x
+ 1;

    blk_4 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x;
    width = 2;
    height = 3;
end
1: begin
    blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
    blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
    blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;

    blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 2;

    width = 3;
    height = 2;
end
2: begin
    blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
    blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
    blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x;
    blk_4 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
    width = 2;
    height = 3;
end
3: begin
    blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
blk_3 = (pos_y * `BLOCKS_WIDE) + pos_x + 2;
blk_4 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 2;

width = 3;
height = 2;

end
endcase
end
`L_BLOCK: begin
    case (rot)
        0: begin
            blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x;
            blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
            blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x;
            blk_4 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x
+ 1;

            width = 2;
            height = 3;

        end
        1: begin
            blk_1 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
            blk_2 = (pos_y * `BLOCKS_WIDE) + pos_x;
            blk_3 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
            blk_4 = (pos_y * `BLOCKS_WIDE) + pos_x + 2;
            width = 3;
            height = 2;

        end
        2: begin
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
blk_1 = (pos_y * `BLOCKS_WIDE) + pos_x + 1;
blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;

blk_3 = ((pos_y + 2) * `BLOCKS_WIDE) + pos_x
+ 1;

blk_4 = (pos_y * `BLOCKS_WIDE) + pos_x;
width = 2;
height = 3;

end

3: begin
    blk_1 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x;
    blk_2 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 1;

    blk_3 = ((pos_y + 1) * `BLOCKS_WIDE) + pos_x
+ 2;

    blk_4 = (pos_y * `BLOCKS_WIDE) + pos_x + 2;
    width = 3;
    height = 2;

end

endcase

end

endcase

end

endmodule

// calc_test_pos_rot: combinational test position chooser

module calc_test_pos_rot(
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
input wire [`MODE_BITS-1:0] mode,
input wire game_clk_rst,
input wire game_clk,
input wire btn_left_en,
input wire btn_right_en,
input wire btn_rotate_en,
input wire btn_down_en,
input wire btn_drop_en,
input wire [`BITS_X_POS-1:0] cur_pos_x,
input wire [`BITS_Y_POS-1:0] cur_pos_y,
input wire [`BITS_ROT-1:0] cur_rot,
output reg [`BITS_X_POS-1:0] test_pos_x,
output reg [`BITS_Y_POS-1:0] test_pos_y,
output reg [`BITS_ROT-1:0] test_rot
);

always @ (*) begin
    test_pos_x = cur_pos_x;
    test_pos_y = cur_pos_y;
    test_rot = cur_rot;

    if (mode == `MODE_PLAY) begin
        if (game_clk) begin
            test_pos_y = cur_pos_y + 1; // move down
        end else if (btn_left_en) begin
            test_pos_x = cur_pos_x - 1; // move left
        end else if (btn_right_en) begin
            test_pos_x = cur_pos_x + 1; // move right
        end else if (btn_rotate_en) begin
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        test_rot = cur_rot + 1; // rotate
    end else if (btn_down_en) begin
        test_pos_y = cur_pos_y + 1; // move down
    end else begin
        // no change
    end
end else if (mode == `MODE_DROP) begin
    if (!game_clk_rst) begin
        test_pos_y = cur_pos_y + 1; // move down
    end
end
end

endmodule

// complete_row: checks board rows for completeness (scans every clk
posedge)

module complete_row(
    input wire                clk,
    input wire                pause,
    input wire [(`BLOCKS_WIDE*`BLOCKS_HIGH)-1:0] fallen_pieces,
    output reg [ `BITS_Y_POS-1:0] row,
    output wire               enabled
);

    initial begin
        row = 0;
    end
end
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// Enabled is high when the current row is complete (all 1s)
assign enabled = &fallen_pieces[row*`BLOCKS_WIDE +:
`BLOCKS_WIDE];

// Increment the row under test when the clock goes high
always @ (posedge clk) begin
    if (!pause) begin
        if (row == `BLOCKS_HIGH - 1) begin
            row <= 0;
        end else begin
            row <= row + 1;
        end
    end
end

endmodule

// debouncer: debounces a raw input relative to a 25 MHz clock
// - Outputs enabled (rising edge) and disabled (falling edge)

module debouncer(
    input wire  raw,
    input wire  clk,
    output wire enabled,
    output wire disabled
);

    reg debounced;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
reg debounced_prev;
reg [16:0] counter; // enough for 125000

initial begin
    debounced = 0;
    debounced_prev = 0;
    counter = 0;
end

// Assumes clk = 25 MHz. For 200 Hz sample: 25_000_000 / 200 =
125000 cycles
always @ (posedge clk) begin
    if (counter == 125000 - 1) begin
        counter <= 0;
        debounced <= raw;
    end else begin
        counter <= counter + 1;
    end

    debounced_prev <= debounced;
end

assign enabled = debounced && !debounced_prev;
assign disabled = !debounced && debounced_prev;

endmodule

// game_clock: produces a 1 Hz game_clk from 25 MHz input
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
module game_clock(  
    input wire clk,  
    input wire rst,  
    input wire pause,  
    output reg game_clk  
);  
  
reg [31:0] counter;  
always @ (posedge clk) begin  
    if (rst) begin  
        counter <= 0;  
        game_clk <= 0;  
    end else begin  
        if (!pause) begin  
            if (counter == 25000000 - 1) begin // 25 MHz -> 1 Hz  
                counter <= 0;  
                game_clk <= 1;  
            end else begin  
                counter <= counter + 1;  
                game_clk <= 0;  
            end  
        end else begin  
            counter <= counter;  
            game_clk <= 0;  
        end  
    end  
end  
  
endmodule
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// randomizer: simple cycling pseudo-random generator (seeded)

module randomizer(
    input wire      clk,
    output reg [2:0] random
);

    initial begin
        random = 1;
    end

    always @ (posedge clk) begin
        if (random == 7) begin
            random <= 1;
        end else begin
            random <= random + 1;
        end
    end

endmodule

// seg_display: multiplexed 7-seg (common anodes active low assumed)

module seg_display(
    input wire      clk,
    input wire [3:0] score_1, // 1's place
    input wire [3:0] score_2, // 10's place
    input wire [3:0] score_3, // 100's place
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
input wire [3:0] score_4, // 1000's place
output reg [7:0] seg,
output reg [3:0] an
);

// Divide the clock so we get a seg_clk that goes
// at a couple hundred Hz for displaying the next digit
reg [17:0] counter;
reg seg_clk;
always @ (posedge clk) begin
    if (counter == 50000 - 1) begin
        counter <= 0;
        seg_clk <= 1;
    end else begin
        counter <= counter + 1;
        seg_clk <= 0;
    end
end

// Which digit we are currently displaying
reg [1:0] digit;

initial begin
    digit = 0;
end

always @ (posedge seg_clk) begin
    digit <= digit + 1;
    if (digit == 0) begin
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        an <= 4'b0111;
        display_digit(score_4);
    end else if (digit == 1) begin
        an <= 4'b1011;
        display_digit(score_3);
    end else if (digit == 2) begin
        an <= 4'b1101;
        display_digit(score_2);
    end else begin
        an <= 4'b1110;
        display_digit(score_1);
    end
end

end

task display_digit;
    input [3:0] d;
    begin
        case (d)
            0: seg <= 8'b11000000;
            1: seg <= 8'b11111001;
            2: seg <= 8'b10100100;
            3: seg <= 8'b10110000;
            4: seg <= 8'b10011001;
            5: seg <= 8'b10010010;
            6: seg <= 8'b10000010;
            7: seg <= 8'b11111000;
            8: seg <= 8'b10000000;
            default: seg <= 8'b10010000;
        endcase
    end
endtask
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        end
    endtask

endmodule // seg_display

// Top-level tetris module

module tetris(
    input wire      clk_too_fast, // e.g., 100 MHz
    input wire      btn_drop,
    input wire      btn_rotate,
    input wire      btn_left,
    input wire      btn_right,
    input wire      btn_down,
    input wire      sw_pause,
    input wire      sw_rst,
    output wire [7:0] rgb,
    output wire      hsync,
    output wire      vsync,
    output wire [7:0] seg,
    output wire [3:0] an
);

    // Clock generation (100 MHz -> 25 MHz)
    // -----
    reg [1:0] clk_div_cnt;
    reg clk; // pixel / game clock domain ~25 MHz
    initial begin
        clk_div_cnt = 0;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
    clk = 0;

end

always @(posedge clk_too_fast) begin
    clk_div_cnt <= clk_div_cnt + 1;
    clk <= clk_div_cnt[1]; // divide by 4 -> 100/4 = 25 MHz
end

// -----
// drop_timer and game state
// -----
reg [31:0] drop_timer;
initial drop_timer = 0;

// random piece source (runs at pixel clk)
wire [`BITS_PER_BLOCK-1:0] random_piece;
randomizer randomizer_ (
    .clk(clk),
    .random(random_piece)
);

// debounced button enables
wire btn_drop_en;
wire btn_rotate_en;
wire btn_left_en;
wire btn_right_en;
wire btn_down_en;

debouncer debouncer_btn_drop_ (
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
.raw(btn_drop),  
.clk(clk),  
.enabled(btn_drop_en),  
.disabled()  
);  
debouncer debouncer_btn_rotate_ (  
.raw(btn_rotate),  
.clk(clk),  
.enabled(btn_rotate_en),  
.disabled()  
);  
debouncer debouncer_btn_left_ (  
.raw(btn_left),  
.clk(clk),  
.enabled(btn_left_en),  
.disabled()  
);  
debouncer debouncer_btn_right_ (  
.raw(btn_right),  
.clk(clk),  
.enabled(btn_right_en),  
.disabled()  
);  
debouncer debouncer_btn_down_ (  
.raw(btn_down),  
.clk(clk),  
.enabled(btn_down_en),  
.disabled()  
);
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// switches
wire sw_pause_en;
wire sw_pause_dis;
wire sw_rst_en;
wire sw_rst_dis;
debouncer debouncer_sw_pause_ (
    .raw(sw_pause),
    .clk(clk),
    .enabled(sw_pause_en),
    .disabled(sw_pause_dis)
);
debouncer debouncer_sw_rst_ (
    .raw(sw_rst),
    .clk(clk),
    .enabled(sw_rst_en),
    .disabled(sw_rst_dis)
);

// fallen pieces memory (1 bit per board cell)
reg [(`BLOCKS_WIDE*`BLOCKS_HIGH)-1:0] fallen_pieces;

// current falling piece state
reg [`BITS_PER_BLOCK-1:0] cur_piece;
reg [`BITS_X_POS-1:0] cur_pos_x;
reg [`BITS_Y_POS-1:0] cur_pos_y;
reg [`BITS_ROT-1:0] cur_rot;

wire [`BITS_BLK_POS-1:0] cur_blk_1;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
wire [`BITS_BLK_POS-1:0] cur_blk_2;  
wire [`BITS_BLK_POS-1:0] cur_blk_3;  
wire [`BITS_BLK_POS-1:0] cur_blk_4;  
wire [`BITS_BLK_SIZE-1:0] cur_width;  
wire [`BITS_BLK_SIZE-1:0] cur_height;
```

```
calc_cur_blk calc_cur_blk_ (  
    .piece(cur_piece),  
    .pos_x(cur_pos_x),  
    .pos_y(cur_pos_y),  
    .rot(cur_rot),  
    .blk_1(cur_blk_1),  
    .blk_2(cur_blk_2),  
    .blk_3(cur_blk_3),  
    .blk_4(cur_blk_4),  
    .width(cur_width),  
    .height(cur_height)  
);
```

```
// VGA display  
vga_display display_ (  
    .clk(clk),  
    .cur_piece(cur_piece),  
    .cur_blk_1(cur_blk_1),  
    .cur_blk_2(cur_blk_2),  
    .cur_blk_3(cur_blk_3),  
    .cur_blk_4(cur_blk_4),  
    .fallen_pieces(fallen_pieces),  
    .rgb(rgb),
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
.hsync(hsync),
.vsync(vsync)
);

// mode FSM and game clock
reg [`MODE_BITS-1:0] mode;
reg [`MODE_BITS-1:0] old_mode;
wire game_clk;
reg game_clk_rst;

game_clock game_clock_ (
    .clk(clk),
    .rst(game_clk_rst),
    .pause(mode != `MODE_PLAY),
    .game_clk(game_clk)
);

// calculate test pos/rot
wire [`BITS_X_POS-1:0] test_pos_x;
wire [`BITS_Y_POS-1:0] test_pos_y;
wire [`BITS_ROT-1:0] test_rot;
calc_test_pos_rot calc_test_pos_rot_ (
    .mode(mode),
    .game_clk_rst(game_clk_rst),
    .game_clk(game_clk),
    .btn_left_en(btn_left_en),
    .btn_right_en(btn_right_en),
    .btn_rotate_en(btn_rotate_en),
    .btn_down_en(btn_down_en),
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
.btn_drop_en(btn_drop_en),
.cur_pos_x(cur_pos_x),
.cur_pos_y(cur_pos_y),
.cur_rot(cur_rot),
.test_pos_x(test_pos_x),
.test_pos_y(test_pos_y),
.test_rot(test_rot)
);

// test block positions from the hypothetical move
wire [`BITS_BLK_POS-1:0] test_blk_1;
wire [`BITS_BLK_POS-1:0] test_blk_2;
wire [`BITS_BLK_POS-1:0] test_blk_3;
wire [`BITS_BLK_POS-1:0] test_blk_4;
wire [`BITS_BLK_SIZE-1:0] test_width;
wire [`BITS_BLK_SIZE-1:0] test_height;
calc_cur_blk calc_test_block_ (
    .piece(cur_piece),
    .pos_x(test_pos_x),
    .pos_y(test_pos_y),
    .rot(test_rot),
    .blk_1(test_blk_1),
    .blk_2(test_blk_2),
    .blk_3(test_blk_3),
    .blk_4(test_blk_4),
    .width(test_width),
    .height(test_height)
);
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// Safe intersection check (avoid indexing with ERR_BLK_POS)
function intersects_fallen_pieces;
    input [7:0] blk1;
    input [7:0] blk2;
    input [7:0] blk3;
    input [7:0] blk4;
    reg b1, b2, b3, b4;
    begin
        b1 = (blk1 == `ERR_BLK_POS) ? 1'b0 : fallen_pieces[blk1];
        b2 = (blk2 == `ERR_BLK_POS) ? 1'b0 : fallen_pieces[blk2];
        b3 = (blk3 == `ERR_BLK_POS) ? 1'b0 : fallen_pieces[blk3];
        b4 = (blk4 == `ERR_BLK_POS) ? 1'b0 : fallen_pieces[blk4];
        intersects_fallen_pieces = b1 | b2 | b3 | b4;
    end
endfunction

wire test_intersects = intersects_fallen_pieces(test_blk_1,
test_blk_2, test_blk_3, test_blk_4);

// Movement tasks
task move_left;
    begin
        if (cur_pos_x > 0 && !test_intersects) begin
            cur_pos_x <= cur_pos_x - 1;
        end
    end
endtask

task move_right;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
begin
    if (cur_pos_x + cur_width < `BLOCKS_WIDE &&
!test_intersects) begin
        cur_pos_x <= cur_pos_x + 1;
    end
end
endtask

task rotate;
begin
    if (cur_pos_x + test_width <= `BLOCKS_WIDE &&
        cur_pos_y + test_height <= `BLOCKS_HIGH &&
!test_intersects) begin
        cur_rot <= cur_rot + 1;
    end
end
endtask

task add_to_fallen_pieces;
begin
    if (cur_blk_1 != `ERR_BLK_POS) fallen_pieces[cur_blk_1]
<= 1;
    if (cur_blk_2 != `ERR_BLK_POS) fallen_pieces[cur_blk_2]
<= 1;
    if (cur_blk_3 != `ERR_BLK_POS) fallen_pieces[cur_blk_3]
<= 1;
    if (cur_blk_4 != `ERR_BLK_POS) fallen_pieces[cur_blk_4]
<= 1;
end
endtask
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
task get_new_block;
    begin
        drop_timer <= 0;
        cur_piece <= random_piece;
        cur_pos_x <= (`BLOCKS_WIDE / 2) - 1;
        cur_pos_y <= 0;
        cur_rot <= 0;
        game_clk_rst <= 1;
    end
endtask

task move_down;
    begin
        if (cur_pos_y + cur_height < `BLOCKS_HIGH &&
!test_intersects) begin
            cur_pos_y <= cur_pos_y + 1;
        end else begin
            add_to_fallen_pieces();
            get_new_block();
        end
    end
endtask

task drop_to_bottom;
    begin
        mode <= `MODE_DROP;
    end
endtask
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// score registers and display
reg [3:0] score_1; // 1's place
reg [3:0] score_2; // 10's place
reg [3:0] score_3; // 100's place
reg [3:0] score_4; // 1000's place

seg_display score_display_ (
    .clk(clk),
    .score_1(score_1),
    .score_2(score_2),
    .score_3(score_3),
    .score_4(score_4),
    .an(an),
    .seg(seg)
);

// complete row module
wire [`BITS_Y_POS-1:0] remove_row_y;
wire remove_row_en;
complete_row complete_row_ (
    .clk(clk),
    .pause(mode != `MODE_PLAY),
    .fallen_pieces(fallen_pieces),
    .row(remove_row_y),
    .enabled(remove_row_en)
);

// remove row task
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
reg [`BITS_Y_POS-1:0] shifting_row;
task remove_row;
    begin
        mode <= `MODE_SHIFT;
        shifting_row <= remove_row_y;
        // increment score
        if (score_1 == 9) begin
            score_1 <= 0;
            if (score_2 == 9) begin
                score_2 <= 0;
                if (score_3 == 9) begin
                    score_3 <= 0;
                    if (score_4 != 9) begin
                        score_4 <= score_4 + 1;
                    end
                end else begin
                    score_3 <= score_3 + 1;
                end
            end else begin
                score_2 <= score_2 + 1;
            end
        end else begin
            score_1 <= score_1 + 1;
        end
    end
endtask

// Initialization
initial begin
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
mode = `MODE_IDLE;
fallen_pieces = 0;
cur_piece = `I_BLOCK;           // visible test piece at
startup
cur_pos_x = 3;
cur_pos_y = 5;
cur_rot = 0;
score_1 = 0;
score_2 = 0;
score_3 = 0;
score_4 = 0;
game_clk_rst = 0;
end

// Start game task
task start_game;
begin
    mode <= `MODE_PLAY;
    fallen_pieces <= 0;
    score_1 <= 0;
    score_2 <= 0;
    score_3 <= 0;
    score_4 <= 0;
    get_new_block();
end
endtask

// game over detection
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
wire game_over = (cur_pos_y == 0) &&
intersects_fallen_pieces(cur_blk_1, cur_blk_2, cur_blk_3, cur_blk_4);

// main FSM
always @ (posedge clk) begin
    // drop timer increments
    if (drop_timer < `DROP_TIMER_MAX) begin
        drop_timer <= drop_timer + 1;
    end

    // clear reset pulse
    game_clk_rst <= 0;

    if (mode == `MODE_IDLE && (sw_rst_en || sw_rst_dis)) begin
        start_game();
    end else if (sw_rst_en || sw_rst_dis || game_over) begin
        mode <= `MODE_IDLE;
        add_to_fallen_pieces();
        cur_piece <= `EMPTY_BLOCK;
    end else if ((sw_pause_en || sw_pause_dis) && mode ==
`MODE_PLAY) begin
        old_mode <= mode;
        mode <= `MODE_PAUSE;
    end else if ((sw_pause_en || sw_pause_dis) && mode ==
`MODE_PAUSE) begin
        mode <= old_mode;
    end else if (mode == `MODE_PLAY) begin
        if (game_clk) begin
            move_down();
        end else if (btn_left_en) begin
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        move_left();
    end else if (btn_right_en) begin
        move_right();
    end else if (btn_rotate_en) begin
        rotate();
    end else if (btn_down_en) begin
        move_down();
    end else if (btn_drop_en && drop_timer ==
`DROP_TIMER_MAX) begin
        drop_to_bottom();
    end else if (remove_row_en) begin
        remove_row();
    end
end else if (mode == `MODE_DROP) begin
    if (game_clk_rst && !sw_pause_en) begin
        mode <= `MODE_PLAY;
    end else begin
        move_down();
    end
end else if (mode == `MODE_SHIFT) begin
    if (shifting_row == 0) begin
        fallen_pieces[0 +: `BLOCKS_WIDE] <= 0;
        mode <= `MODE_PLAY;
    end else begin
        fallen_pieces[shifting_row*`BLOCKS_WIDE +:
`BLOCKS_WIDE] <= fallen_pieces[(shifting_row - 1)*`BLOCKS_WIDE +:
`BLOCKS_WIDE];
        shifting_row <= shifting_row - 1;
    end
end
end
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
end

endmodule

// vga_display: produces hsync/vsync and pixel colors for board +
falling piece

// vga_display: produces hsync/vsync and pixel colors for board +
falling piece

// NOTE: Declarations are all at module scope (no declarations inside
always blocks)

module vga_display(
    input wire                clk,
    input wire [`BITS_PER_BLOCK-1:0] cur_piece,
    input wire [`BITS_BLK_POS-1:0] cur_blk_1,
    input wire [`BITS_BLK_POS-1:0] cur_blk_2,
    input wire [`BITS_BLK_POS-1:0] cur_blk_3,
    input wire [`BITS_BLK_POS-1:0] cur_blk_4,
    input wire [(`BLOCKS_WIDE*`BLOCKS_HIGH)-1:0] fallen_pieces,
    output reg [7:0]          rgb,
    output wire              hsync,
    output wire              vsync
);

    // counters
    reg [10:0] counter_x;
    reg [10:0] counter_y;

    // temporary vars for block index calculation (module scope)
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
reg [9:0] bx;
reg [9:0] by;
reg [7:0] cur_blk_index;

initial begin
    counter_x = 0;
    counter_y = 0;
    bx = 0;
    by = 0;
    cur_blk_index = `ERR_BLK_POS;
end

// HSYNC / VSYNC (active low in this mapping)
assign hsync = ~(counter_x >= (`PIXEL_WIDTH + `HSYNC_FRONT_PORCH)
&&
                    counter_x < (`PIXEL_WIDTH + `HSYNC_FRONT_PORCH +
`HSYNC_PULSE_WIDTH));
    assign vsync = ~(counter_y >= (`PIXEL_HEIGHT +
`VSYNC_FRONT_PORCH) &&
                    counter_y < (`PIXEL_HEIGHT + `VSYNC_FRONT_PORCH
+ `VSYNC_PULSE_WIDTH));

// compute whether inside board area (use comparisons)
wire inside_board = (counter_x >= `BOARD_X) && (counter_y >=
`BOARD_Y) &&
                    (counter_x < (`BOARD_X + `BOARD_WIDTH)) &&
(counter_y < (`BOARD_Y + `BOARD_HEIGHT));

// compute current block index safely (only used when
inside_board)
always @ (*) begin
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
    if (inside_board) begin
        // integer division by BLOCK_SIZE (caution: synthesis
will synth)
        bx = (counter_x - `BOARD_X) / `BLOCK_SIZE;
        by = (counter_y - `BOARD_Y) / `BLOCK_SIZE;
        cur_blk_index = bx + (by * `BLOCKS_WIDE);
    end else begin
        bx = 0;
        by = 0;
        cur_blk_index = `ERR_BLK_POS;
    end
end

// pixel color logic
always @ (*) begin
    if (inside_board) begin
        // thin border
        if ((counter_x == `BOARD_X) || (counter_x == (`BOARD_X +
`BOARD_WIDTH - 1)) ||
            (counter_y == `BOARD_Y) || (counter_y == (`BOARD_Y +
`BOARD_HEIGHT - 1))) begin
            rgb = `WHITE;
        end else begin
            // falling piece pixels
            if ((cur_blk_index == cur_blk_1) ||
                (cur_blk_index == cur_blk_2) ||
                (cur_blk_index == cur_blk_3) ||
                (cur_blk_index == cur_blk_4)) begin
                case (cur_piece)
                    `EMPTY_BLOCK: rgb = `GRAY;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
`I_BLOCK: rgb = `CYAN;
`O_BLOCK: rgb = `YELLOW;
`T_BLOCK: rgb = `PURPLE;
`S_BLOCK: rgb = `GREEN;
`Z_BLOCK: rgb = `RED;
`J_BLOCK: rgb = `BLUE;
`L_BLOCK: rgb = `ORANGE;
default: rgb = `GRAY;

    endcase
end else begin
    if (cur_blk_index != `ERR_BLK_POS &&
fallen_pieces[cur_blk_index])
        rgb = `WHITE;
    else
        rgb = `GRAY;
    end
end
end else begin
    // outside board -> background
    rgb = `BLACK;
end
end

// counters for sync generation: iterate through entire frame +
porches
always @ (posedge clk) begin
    if (counter_x >= `PIXEL_WIDTH + `HSYNC_FRONT_PORCH +
`HSYNC_PULSE_WIDTH + `HSYNC_BACK_PORCH - 1) begin
        counter_x <= 0;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
        if (counter_y >= `PIXEL_HEIGHT + `VSYNC_FRONT_PORCH +
`VSYNC_PULSE_WIDTH + `VSYNC_BACK_PORCH - 1) begin
            counter_y <= 0;
        end else begin
            counter_y <= counter_y + 1;
        end
    end else begin
        counter_x <= counter_x + 1;
    end
end
end

endmodule
```

Implementation Constraints File:

```
# tetris.ucf
# Constraint file for the tetris top-level module
# Map top-level ports to physical FPGA pins (edit if your board
pinout differs)

# System clock input (assumed 100 MHz)
NET "clk_too_fast" LOC = "V10" | IOSTANDARD = "LVCMOS33";
NET "clk_too_fast" TNM_NET = sys_clk_pin;
TIMESPEC "TS_sys_clk_pin" = PERIOD "sys_clk_pin" 10 ns HIGH 50%;

# 7-segment display (seg[0]..seg[7])
NET "seg<0>"          LOC = "T17" | IOSTANDARD = "LVCMOS33";
NET "seg<1>"          LOC = "T18" | IOSTANDARD = "LVCMOS33";
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
NET "seg<2>"      LOC = "U17" | IOSTANDARD = "LVCMOS33";
NET "seg<3>"      LOC = "U18" | IOSTANDARD = "LVCMOS33";
NET "seg<4>"      LOC = "M14" | IOSTANDARD = "LVCMOS33";
NET "seg<5>"      LOC = "N14" | IOSTANDARD = "LVCMOS33";
NET "seg<6>"      LOC = "L14" | IOSTANDARD = "LVCMOS33";
NET "seg<7>"      LOC = "M13" | IOSTANDARD = "LVCMOS33";

# 7-seg anodes (an[0]..an[3])
NET "an<0>"       LOC = "N16" | IOSTANDARD = "LVCMOS33";
NET "an<1>"       LOC = "N15" | IOSTANDARD = "LVCMOS33";
NET "an<2>"       LOC = "P18" | IOSTANDARD = "LVCMOS33";
NET "an<3>"       LOC = "P17" | IOSTANDARD = "LVCMOS33";

# Switches (sw_pause, sw_rst)
NET "sw_pause"    LOC = "T10" | IOSTANDARD = "LVCMOS33";
NET "sw_rst"      LOC = "T5"  | IOSTANDARD = "LVCMOS33";

# Buttons (btn_drop, btn_rotate, btn_left, btn_down, btn_right)
NET "btn_drop"    LOC = "B8"  | IOSTANDARD = "LVCMOS33";
NET "btn_rotate"  LOC = "A8"  | IOSTANDARD = "LVCMOS33";
NET "btn_left"    LOC = "C4"  | IOSTANDARD = "LVCMOS33";
NET "btn_down"    LOC = "C9"  | IOSTANDARD = "LVCMOS33";
NET "btn_right"   LOC = "D9"  | IOSTANDARD = "LVCMOS33";

# VGA outputs (rgb[0]..rgb[7])
NET "rgb<0>"      LOC = "U7"   | IOSTANDARD = "LVCMOS33";
NET "rgb<1>"      LOC = "V7"   | IOSTANDARD = "LVCMOS33";
NET "rgb<2>"      LOC = "N7"   | IOSTANDARD = "LVCMOS33";
NET "rgb<3>"      LOC = "P8"   | IOSTANDARD = "LVCMOS33";
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

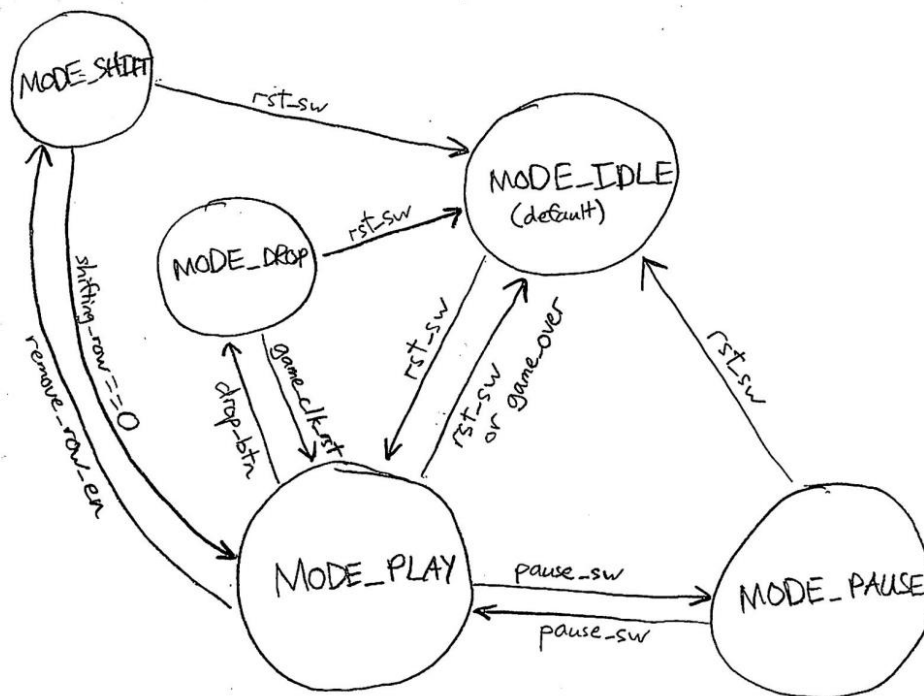
Nosheen Fatima (411289)

Ujala Kiran (432071)

```
NET "rgb<4>"      LOC = "T6" | IOSTANDARD = "LVCMOS33";
NET "rgb<5>"      LOC = "V6" | IOSTANDARD = "LVCMOS33";
NET "rgb<6>"      LOC = "R7" | IOSTANDARD = "LVCMOS33";
NET "rgb<7>"      LOC = "T7" | IOSTANDARD = "LVCMOS33";

# VGA sync
NET "hsync"        LOC = "N6" | IOSTANDARD = "LVCMOS33";
NET "vsync"        LOC = "P7" | IOSTANDARD = "LVCMOS33";
```

State Machine Diagram:



Test Bench:

```
`timescale 1ns / 1ps

module tb_tetris;
```

DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// Clock & Inputs

reg clk_too_fast;    // 100 MHz
reg btn_drop;
reg btn_rotate;
reg btn_left;
reg btn_right;
reg btn_down;
reg sw_pause;
reg sw_rst;

// Outputs (ignore seg/an)

wire [7:0] rgb;
wire hsync;
wire vsync;

// DUT Instantiation

tetris dut (
    .clk_too_fast(clk_too_fast),
    .btn_drop(btn_drop),
    .btn_rotate(btn_rotate),
    .btn_left(btn_left),
    .btn_right(btn_right),
    .btn_down(btn_down),
    .sw_pause(sw_pause),
    .sw_rst(sw_rst),
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
.rgb(rgb),
.hsinc(hsync),
.vsync(vsync),
.seg(),    // intentionally left unconnected
.an()      // intentionally left unconnected
);

// 100 MHz Clock

initial begin
    clk_too_fast = 0;
    forever #5 clk_too_fast = ~clk_too_fast;
end

// Dumpfile

initial begin
    $dumpfile("tetris_tb.vcd");
    $dumpvars(0, tb_tetris);
end

// Stimulus

initial begin
    // defaults
    btn_drop    = 0;
    btn_rotate  = 0;
    btn_left    = 0;
    btn_right   = 0;
    btn_down    = 0;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
sw_pause    = 0;
sw_rst      = 0;

// Reset
#100;
sw_rst = 1;
#200;
sw_rst = 0;

// Let game start
#2_000_000;

// Move left
btn_left = 1;
#50_000;
btn_left = 0;

// Move right
#500_000;
btn_right = 1;
#50_000;
btn_right = 0;

// Rotate
#500_000;
btn_rotate = 1;
#50_000;
btn_rotate = 0;
```

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

Ujala Kiran (432071)

```
// Soft drop
#500_000;
btn_down = 1;
#100_000;
btn_down = 0;

// Hard drop
#500_000;
btn_drop = 1;
#50_000;
btn_drop = 0;

// Pause
#1_000_000;
sw_pause = 1;
#1_000_000;
sw_pause = 0;

// Run longer
#5_000_000;
$finish;

end

endmodule
```

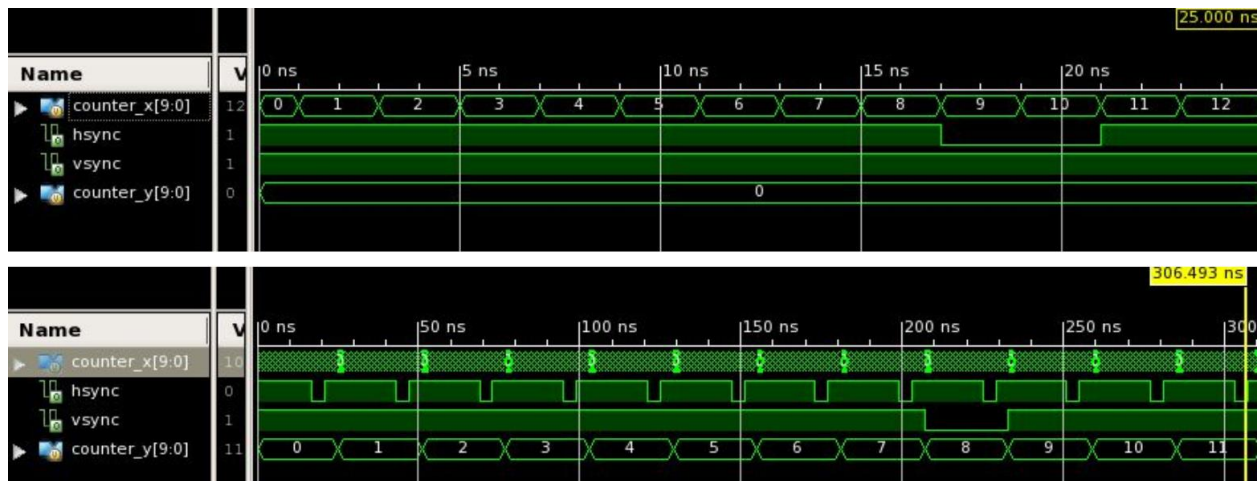
Submitted By:

Nosheen Fatima (411289)

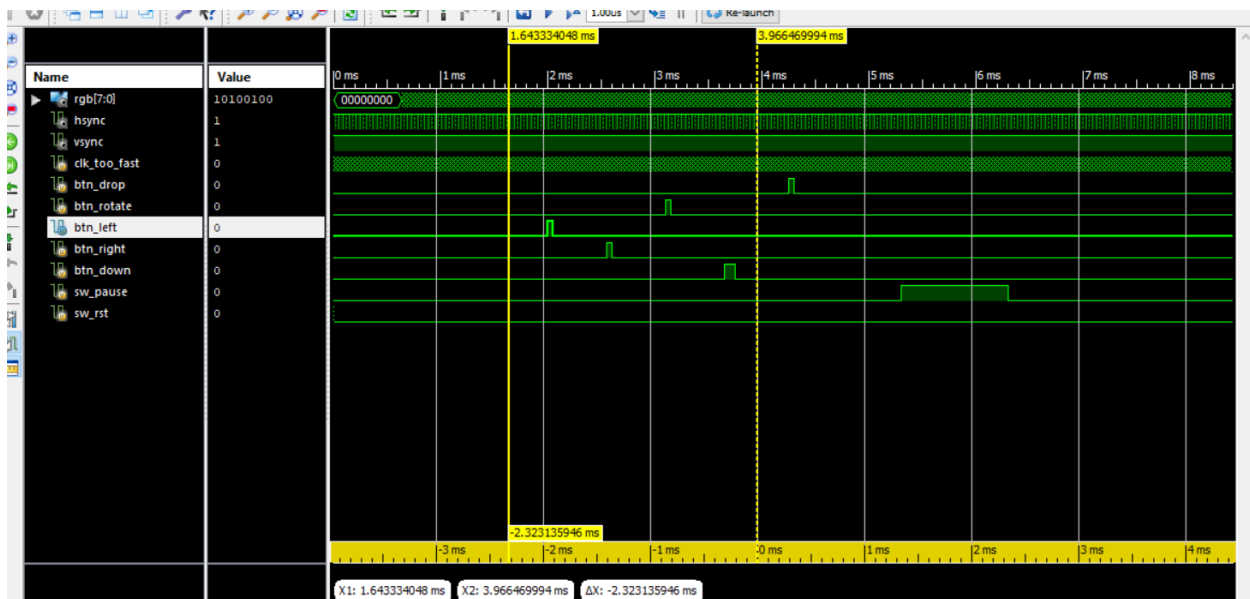
Ujala Kiran (432071)

Output:

Simulation:



This is the waveform output for the simulation of VGA controller. The top image shows the first 25 cycles, which is up until the end of horizontal line. The bottom image shows all 311 cycles, which is the entire first video frame. In the top image, counter_x goes from 0 to 7 and all the rgb signal should be busy outputting pixels. Then when counter_x is 8, this is the front porch for hsync. When counter_x is 9 and 10, this is the hsync pulse. After, there is another lull for back porch before the net line starts. A similar scenario occurs in the bottom image with the relationship between vsync and counter_y signals.



Simulation of tetris game and in Xilinx to show results in wave

**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING COLLEGE OF
E&ME, NUST, RAWALPINDI**

Submitted By:

Nosheen Fatima (411289)

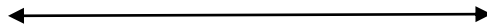
Ujala Kiran (432071)

FPGA Implementation:

Video Link:

https://drive.google.com/file/d/1VF4D_ibsAVxEKGFwRufqkWpreM0cZoqE/view?usp=sharing

<https://drive.google.com/file/d/1Wwr5dhBKknvjycevQKvxzEXLD7MHaHW-/view?usp=sharing>



Conclusion:

In this project, we successfully designed and implemented a Tetris game on the Nexys 3 FPGA board using Verilog. The game correctly handles block movement, scoring, and VGA display by using proper clock control and state-based logic. This project helped us understand FPGA-based system design, VGA interfacing, and real-time hardware implementation in a practical way. Overall, the project achieved its objectives and worked reliably on hardware.
